

Real-Time Collaborative Text Editor - Interview Guide

Design: Google Docs / Notion / Confluence

For 12+ Year Experienced Engineers

Quick Navigation

1. [Interview Format](#)
 2. [HLD Interview \(45-60 min\)](#)
 3. [LLD Interview \(45-60 min\)](#)
 4. [Common Questions & Answers](#)
 5. [Cheat Sheet](#)
-

Interview Format

HLD (High-Level Design) Interview

Focus: Architecture, real-time collaboration, conflict resolution, distributed systems **Time:** 45-60 minutes **Typical at:** FAANG, Senior/Staff/Principal Engineer roles

LLD (Low-Level Design) Interview

Focus: OT vs CRDT algorithms, WebSocket management, versioning, concurrency **Time:** 45-60 minutes **Unique challenge:** This problem requires explaining TWO algorithms — OT and CRDT

HLD Interview (45-60 min)

Phase 1: Requirements Gathering (5-10 min)

Interviewer: "Design Google Docs"

Your Response — *Say this out loud, confidently:*

"This is a fascinating problem. Before I jump in, let me ask a few clarifying questions — because collaborative text editing has some very tricky algorithmic challenges underneath."

Functional Requirements — Ask these:

Q: Should users be able to create, edit, and delete documents?

A: Yes — full CRUD on documents.

Q: Can multiple users edit the same document simultaneously?

A: Yes — that's the core feature.

Q: Should users see each other's changes in real time?

A: Yes — near real-time, like Google Docs.

Q: Do we need to show cursor positions of other users?

A: Yes – with color-coded cursors per user.

Q: Do we need document versioning / history?

A: Yes – users should be able to roll back to previous versions.

Q: Rich text (bold, images, tables) or plain text only?

A: Start with plain text, mention rich text as extension.

Q: Do we need offline editing (sync when reconnected)?

A: Nice to have – mention CRDT handles this naturally.

Non-Functional Requirements – Ask these:

Q: Scale – how many users and documents?

A: Millions of users, billions of documents.

Q: How many users can edit one document simultaneously?

A: Up to 100 concurrent editors per document.

Q: Latency requirement for real-time sync?

A: < 100ms for edit propagation between users.

Q: Availability vs Consistency trade-off?

A: High availability overall.

Strong consistency within an active editing session.

Eventual consistency for non-collaborative (solo) editing.

Q: Durability?

A: No data loss – auto-save every 10–20 seconds minimum.

Write this summary on the whiteboard:

GOOGLE DOCS REQUIREMENTS SUMMARY	
Functional	Non-Functional
Create / Edit / Delete docs	Millions of users
Real-time collaborative edit	Billions of documents
See other user's changes	Up to 100 concurrent editors/doc
Cursor position of others	< 100ms edit propagation latency
Document versioning	High availability
Auto-save	Strong consistency in active session

Phase 2: Capacity Estimation (5 min)

"Let me quickly size the system so we know what we're building for."

Users and Documents:

```
Users:      500M registered, 100M DAU
Documents:  10 billion total
Active docs: ~5M being edited at any point in time
```

Edit Events (the tricky part):

```
Per active document: ~5 edits/sec (keystrokes)
Total edit events:    5M docs × 5 edits/sec = 25M events/sec
```

That's 25 MILLION edit events per second.
This is why we can't use normal REST APIs here.

Storage:

```
Document content (text):
  Avg doc size = 50 KB
  10B docs × 50KB = 500 TB (document content)

Operation log (edit events):
  Each operation = ~50 bytes
  25M ops/sec × 86,400 sec = 2.16 trillion ops/day
  → We DON'T store all of these permanently
  → Only keep during active session, then reconcile

Versioning snapshots:
  ~10 versions per doc × 50KB = 500KB per doc
  10B docs × 500KB = 5 PB (all version snapshots)
```

Connections:

```
100M DAU, ~30% active at peak = 30M concurrent WebSocket connections
At 100K conns/server → 300 WebSocket servers needed
```

Visual Summary (draw this):

CAPACITY AT A GLANCE

Edit events	25M/sec (can't use REST – need WebSocket)
Active docs	5M simultaneously being edited
WS connections	30M concurrent
Content storage	500 TB (S3/Blob)
Version storage	5 PB (S3, lifecycle to Glacier for old)
Operation logs	Transient (kept during session, then deleted)

Key insight: 25M edit events/sec is the design-defining number.
This forces: WebSocket + in-memory canonical copy + async persistence

Phase 3: Core Entities (3 min)

"Before the architecture, let me call out the four core entities — this shapes the data model."

CORE ENTITIES	
User / Editor	Who is making changes
Document	The file being edited (metadata + content)
Operation	A single edit event (insert/delete at position)
Version	A snapshot of the document at a point in time
Cursor	Real-time cursor position of each active user

Relationship:

User — creates/edits —> Document
Document — has many —> Versions (snapshots)
Document — has many —> Operations (transient, during session)
User — has a —> Cursor (per document, real-time)

Phase 4: API Design (5 min)

"Let me define the key APIs. Most are standard REST — but the editing API is different."

API ENDPOINTS	
Method	Endpoint + Purpose
POST	/documents Create new document → returns documentId
GET	/documents/{docId} Fetch document (read-only, latest version)
DELETE	/documents/{docId}

	Soft delete document
GET	/documents/{docId}/versions List all versions of a document
GET	/documents/{docId}/versions/{versionId} Fetch a specific historical version
WebSocket	ws://editor/{docId}?userId={userId} Real-time editing channel – ALL edit operations go through here, NOT via REST

Why WebSocket for editing?

REST: stateless, one request = one response, ~50ms overhead/call

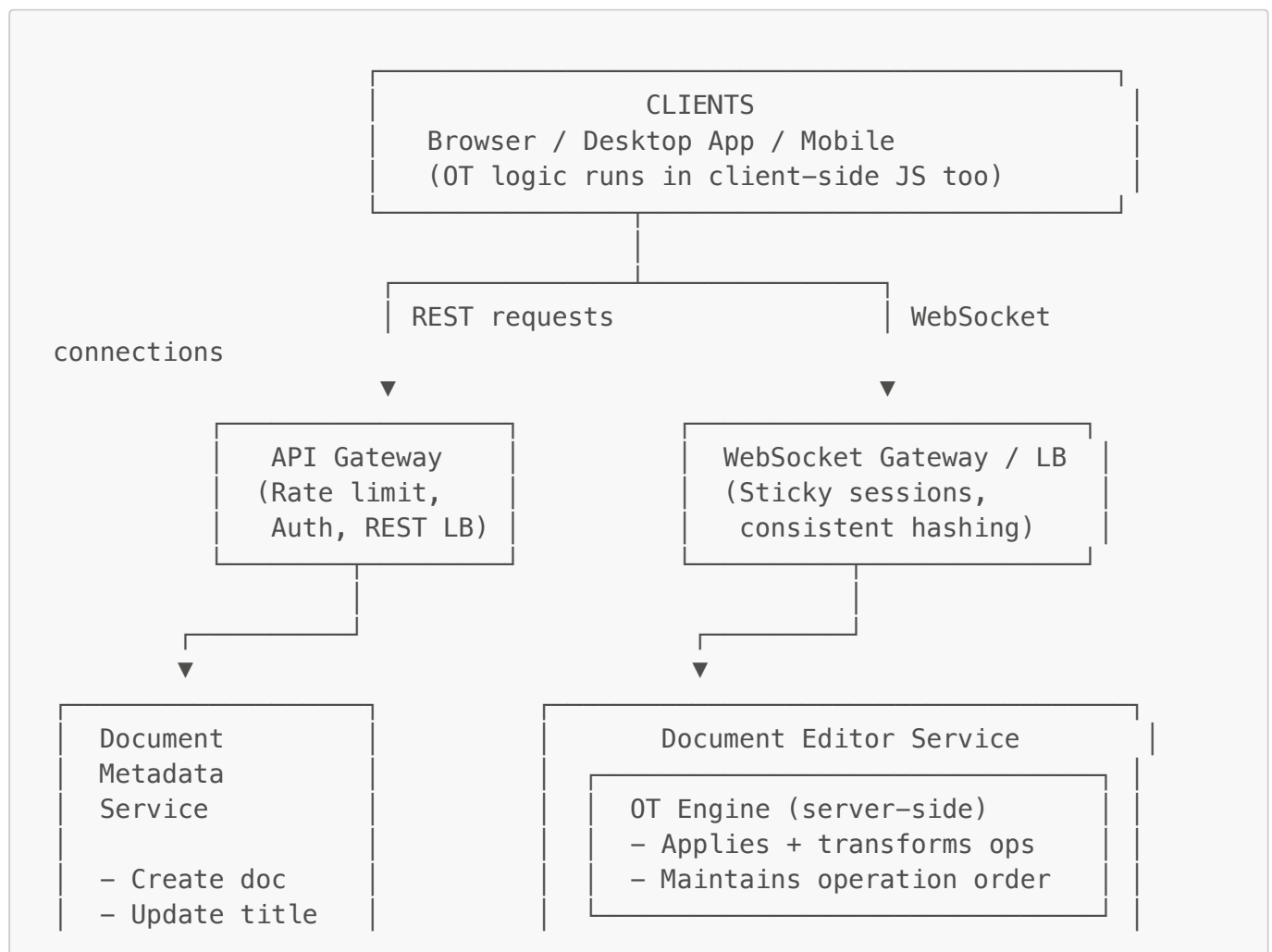
WebSocket: persistent bidirectional connection, ~1ms overhead

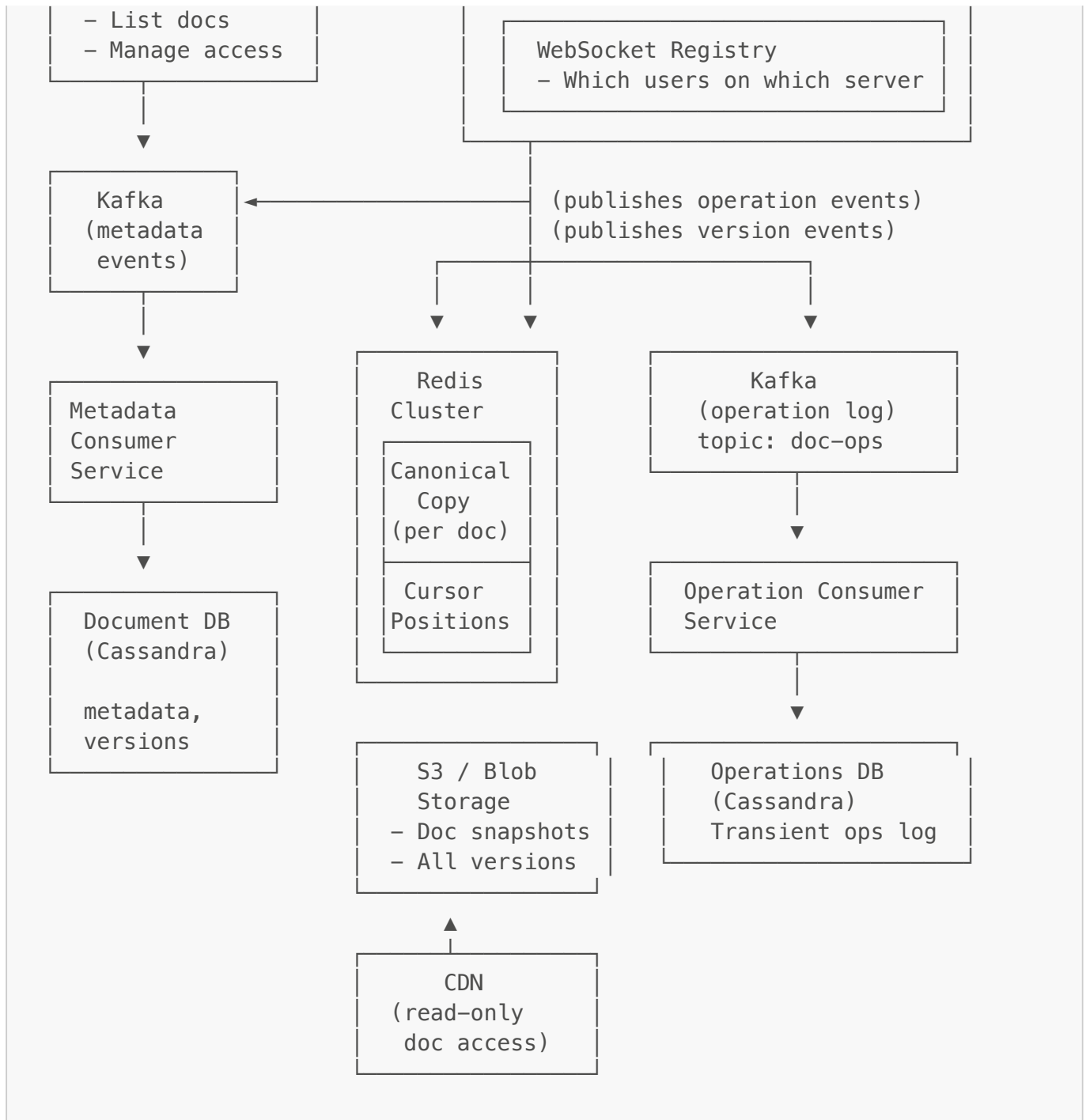
At 25M edit events/sec → REST would collapse under the overhead

Phase 5: High-Level Architecture (15 min)

"Now let me draw the architecture. The system has two distinct modes: read/write metadata (standard REST) and real-time editing (WebSocket). I'll design both paths."

Draw this on the whiteboard:





Walk through each component (2 min each):

1. API Gateway (REST path)

"Handles all non-editing requests: creating documents, fetching metadata, listing versions. Standard rate limiting, auth, load balancing. Nothing special here."

2. WebSocket Gateway (editing path)

"This is different from the REST LB. We use sticky sessions via consistent hashing on documentId. All users editing document X land on the SAME editor server. This is CRITICAL for the OT algorithm — all ops for a doc must go through one server."

3. Document Editor Service + OT Engine

"The heart of the system. Receives edit operations via WebSocket, runs them through the OT engine to resolve conflicts, updates the Redis canonical copy, broadcasts to all connected users, and publishes to Kafka for persistence."

4. Redis Canonical Copy

"This is the live, in-memory version of the document being edited. Think of it as the whiteboard everyone is writing on together. TTL-based: exists only while the document has active editors. When last user leaves, the final version gets flushed to S3."

5. S3 / Blob Storage

"Permanent home for all document versions. Auto-save every 10-20 seconds creates a new snapshot. CDN sits in front for read-only access."

6. Kafka + Operation Consumer

"All edit operations are published to Kafka for durability. Consumer persists them to Cassandra. Used by the reconciliation job at session end."

7. Document DB (Cassandra)

"Stores document metadata AND the operation log. Operation log is transient — deleted after session reconciliation."

Phase 6: Deep Dive — The 3 Approaches to Collaborative Editing

"This is the most important part of this interview. Let me walk you through three approaches — starting with the naive ones, then the correct one."

Approach 1: File Replacement (Naive — Don't Use)

Every keystroke:

```
graph TD
    A[User A types "R"] --> B[Send ENTIRE document to server]
    B --> C[Server saves entire document]
    C --> D[Push entire document to all other users]
    D --> E[All clients re-render full document]
```

Problems:

Problem 1: Payload size

$50\text{KB document} \times 25\text{M events/sec} = 1.25\text{ PB/sec of traffic!}$

Completely infeasible.

Problem 2: Request volume

Each keystroke = 1 full HTTP round trip

At 100 concurrent editors $\times$ 5 keystrokes/sec = 500 req/sec

Per document $\times$ 5M active docs = 2.5 billion req/sec

No server farm can handle this.

VERDICT: Reject. Only mention to show you know the evolution.

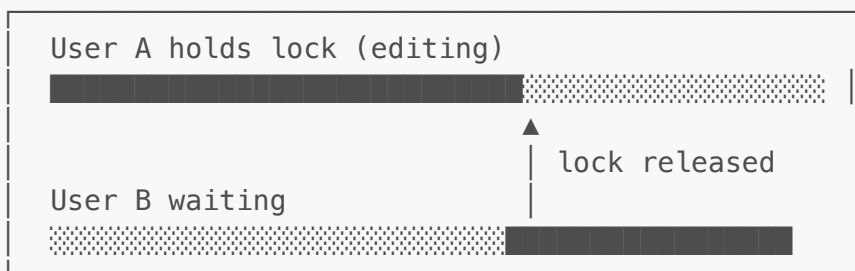
Approach 2: Locking Protocol (Better, But Still Wrong)

Pessimistic Locking:

User A requests edit $\rightarrow$ acquires lock on document

User B tries to edit $\rightarrow$ BLOCKED until A releases lock

A finishes $\rightarrow$ releases lock $\rightarrow$ B can now edit



Real-world analogy: git commit – only one person writes at a time.

Used in: banking, ticket booking (pessimistic = safe but slow)

Optimistic Locking:

Both users edit simultaneously on their local copies.

When either submits $\rightarrow$ check for conflict.

If conflict $\rightarrow$ user must resolve manually, then re-submit.

Real-world analogy: git merge – conflicts surfaced at merge time.

Used in: version control systems.

Problems with both for real-time collaboration:

Pessimistic: No simultaneous editing – defeats the purpose

Optimistic: Manual conflict resolution – terrible UX

VERDICT: Reject. Neither gives seamless real-time editing.

Approach 3: Delta-Based Updates + Conflict Resolution (Correct)

"Instead of sending the whole file, we send only the CHANGE — the delta. And instead of locking, we use an algorithm to automatically resolve conflicts."

Delta-based update:

Document: "WORLD"

User A inserts "H" at position 0

OLD approach: send "HWORLD" (entire file)

NEW approach: send Operation { type: INSERT, pos: 0, char: "H" }

Operation payload: ~50 bytes (vs 50KB full file) 1000× smaller payload Enables 25M ops/sec on normal hardware

But there's still a conflict problem:

User A: INSERT "H" at position 0 → "H" + "WORLD" = "HWORLD"

User B: INSERT "D" at position 5 → "WORLD" + "D" = "WORLD D"

If both applied naively:

A's view: "H" at 0, "D" at 5 → "HWORLD D"?

B's view: "D" at 5, "H" at 0 → "HWORLD D"?

Maybe OK here... but in the general case, positions shift.

The real problem — positions shift when another user inserts:

Doc: "BC" (positions: B=0, C=1)

Alice inserts "A" at position 0 → "ABC"

Bob inserts "D" at position 2 → "BCD"

If Bob's op (insert D at pos 2) applied to Alice's "ABC":

D at position 2 → "ABDC" ← WRONG! should be "ABCD"

Because Alice's insert shifted all positions by 1!

→ We need an algorithm that adjusts positions based on concurrent ops.

→ Two solutions: OT (Operational Transformation) and CRDT.

Phase 7: Deep Dive — OT Algorithm (What Google Uses)

"OT is what Google Docs actually uses. Let me walk through how it works."

The Core Insight:

OPERATIONAL TRANSFORMATION

Instead of applying operations blindly,
TRANSFORM each operation to account for
concurrent operations that happened first.

$\text{transform}(\text{op1}, \text{op2}) \rightarrow \text{op1}'$
 $\text{op1}'$ is op1 adjusted for the effect of op2

Step-by-Step Example:

Initial state: "BC" (B at index 0, C at index 1)

Alice: INSERT "A" at position 0

Bob: INSERT "D" at position 2 (same initial state)

Step 1: Alice applies her op locally

Alice's doc: "A" + "BC" = "ABC"

Alice sends: { INSERT, pos=0, char="A" } to server

Step 2: Bob applies his op locally

Bob's doc: "BC" + "D" = "BCD"

Bob sends: { INSERT, pos=2, char="D" } to server

Step 3: Server receives both ops (assume Alice first)

Server has: "ABC" (after Alice's op)

Bob's op arrives: { INSERT, pos=2, char="D" }

OT TRANSFORM:

Bob's op was based on "BC" (Alice hadn't inserted)
 But now Alice has inserted at pos 0
 → Every position AFTER pos 0 shifts by +1
 → Bob's pos=2 becomes pos=3
 → Transformed op: { INSERT, pos=3, char="D" }

Step 4: Apply transformed Bob's op to "ABC"

"ABC" + INSERT "D" at pos 3 = "ABCD" ✓

Step 5: Broadcast TRANSFORMED ops to both clients

Alice receives: { INSERT, pos=3, char="D" } → "ABCD" ✓

Bob receives: { INSERT, pos=0, char="A" } → "ABCD" ✓

Both clients converge to "ABCD" ✓

OT Transform Rules:

OT TRANSFORM RULES

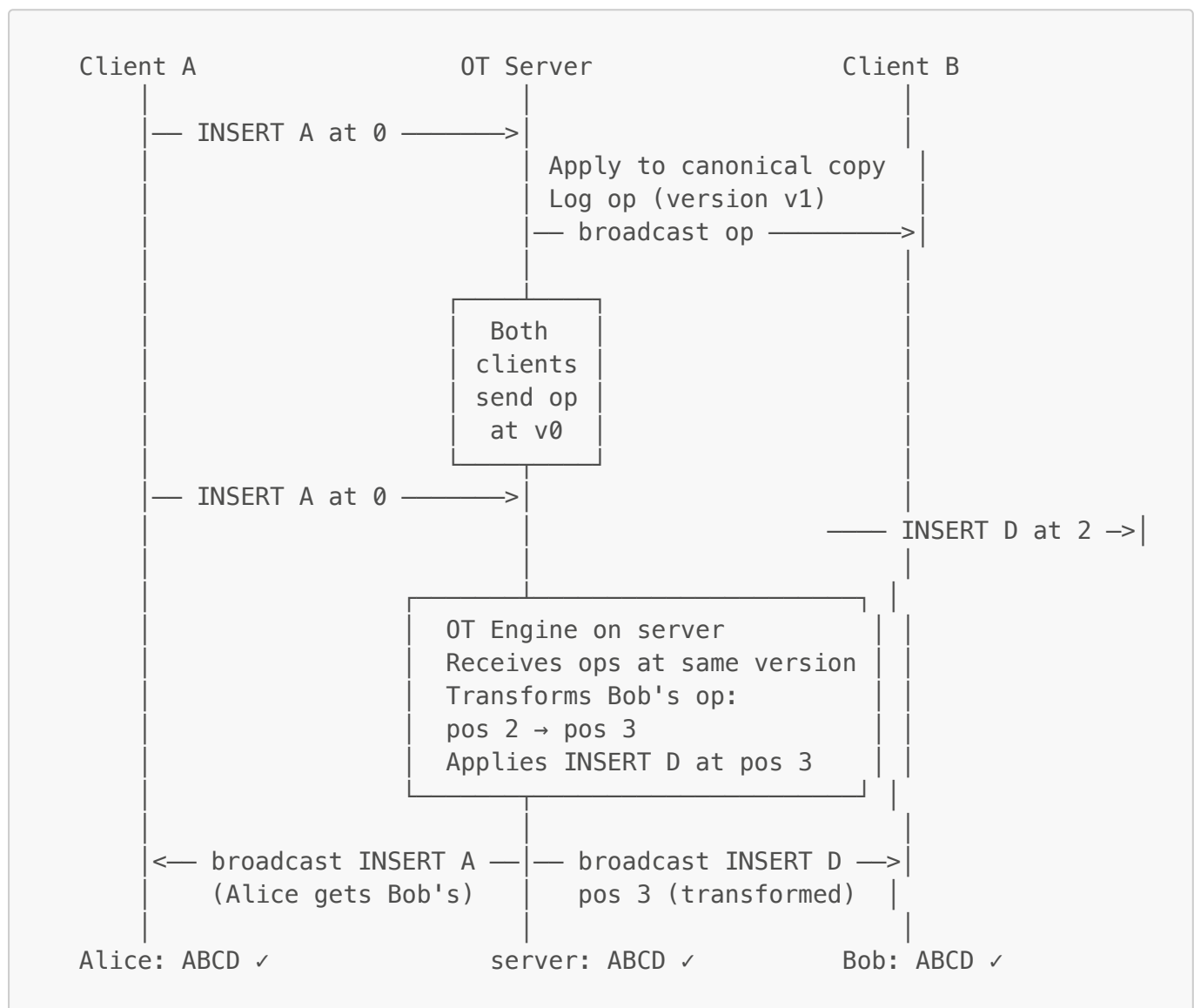
```

When op2 (arrived first) is INSERT at pos P:
  If op1.pos > P:  op1.pos = op1.pos + 1  (shift right)
  If op1.pos < P:  op1.pos unchanged
  If op1.pos = P:  break tie by user ID (deterministic)

When op2 is DELETE at pos P:
  If op1.pos > P:  op1.pos = op1.pos - 1  (shift left)
  If op1.pos = P:  op1 = no-op (deleted char no longer exists)
  If op1.pos < P:  op1.pos unchanged

```

OT Architecture:



OT Limitations — Be upfront:

OT LIMITATIONS

1. All operations MUST go through a SINGLE server

- Users editing doc X must ALL be on the same server
 - Requires sticky routing (consistent hashing by docId)
2. Complex to implement correctly
 - Edge cases: 3+ concurrent users, nested transformations
 - Google's paper on OT is notoriously hard to implement
 3. Doesn't work well for offline editing
 - User edits offline → reconnects → massive conflict
 - CRDT handles this better

Phase 8: CRDT — The Alternative (Notion, Figma Use This)

"The second algorithm is CRDT — Conflict-free Replicated Data Types. Instead of transforming operations, it uses a smarter data structure where positions NEVER conflict."

The Core Insight:

CRDT INSIGHT

OT Problem: Integer positions shift when insertions happen.

CRDT Solution: Don't use integer positions!
Use FRACTIONAL positions that never collide.

Between any two fractions, you can always find a new fraction.
0.25 and 0.5 → insert at 0.375 (never conflicts with others)

CRDT Example:

Initial doc: "BC"

Character	Position (fractional)
B	0.50
C	0.75

Alice inserts "A" BEFORE B:

- Needs a position between 0 and 0.50
- Assigns position 0.25

A	0.25
B	0.50
C	0.75

Bob inserts "D" AFTER C at the same time:

- Needs a position > 0.75
- Assigns position 0.875

B	0.50
C	0.75
D	0.875

Merge both:

A	0.25	← from Alice
B	0.50	
C	0.75	
D	0.875	← from Bob

Sort by position → ABCD ✓

No transformation needed!

Alice's op and Bob's op don't interfere with each other.

Merge is just: sort all characters by their fractional position.

OT vs CRDT Comparison:

OT vs CRDT		
Dimension	OT	CRDT
Core idea	Transform positions	Fractional positions
Server required	YES (single server)	NO (peer-to-peer OK)
Offline edit	Difficult	Natural (merge on sync)
Complexity	High (edge cases)	Medium (storage grows)
Storage	Low	Higher (positions store)
Used by	Google Docs	Figma, Notion, Atom
Scalability	Centralized	Distributed

For this interview: implement OT (Google's approach)

Mention CRDT as the alternative with its trade-offs.

Phase 9: Scalability & Trade-offs (5-10 min)

1. Sticky Routing for OT

OT requires all users on same doc → same server.

Solution: Consistent hashing on documentId

DocumentId hash → hash ring → WS Server assignment

WebSocket Load Balancer

doc-123 → hash(123) % N → WS Server 3

doc-456 → hash(456) % N → WS Server 7

All users of doc-123 connect to WS Server 3

→ OT engine on Server 3 has all ops

→ No cross-server coordination needed

What if WS Server 3 fails?

→ Rehash to new server

→ New server fetches canonical copy from Redis

→ All users reconnect (brief blip, < 1 second)

2. Redis Canonical Copy TTL

Document opens: Redis stores canonical copy

Key: canonical:{docId}

Value: full document text

TTL: refreshed as long as ≥1 active editor

When last editor disconnects:

→ Save final version to S3

→ Delete Redis key (TTL expires or explicit delete)

3. Auto-Save and Versioning

Every 10–20 seconds during active editing:

Editor Service reads Redis canonical copy

→ Saves snapshot to S3 as new minor version

→ Publishes version event to Kafka

→ Metadata Consumer updates version table in DB

Session end reconciliation:

Reconciliation Job:

1. Fetch base version from S3 (before session)

2. Fetch all ops from Operations DB (Cassandra)
3. Replay all ops on base version
4. Save as new MAJOR version to S3
5. Delete all minor versions from session
6. Delete all ops from Cassandra (session cleanup)

Version progression:

v10 (base) → v10.1 → v10.2 → ... → v10.50 (minor)

↓

v11 (major, reconciled)

v10.1...v10.50 DELETED All ops DELETED

Trade-offs Table:

Decision	Why We Chose It	The Trade-off
WebSocket over REST	25M ops/sec — REST would collapse	Stateful servers, sticky routing needed
OT over CRDT	Simpler server, Google's proven approach	Single server per doc, no offline editing
Redis for canonical copy	Microsecond read/write, in-memory speed	Volatile — must flush to S3 on disconnect
Delta ops not full file	1000x less bandwidth	Conflict resolution algorithm needed
Reconciliation job	Clean up minor versions, save storage	Eventual consistency on version history
Cassandra for ops log	High write throughput, time-series	Transient data — deleted after reconcile

LLD Interview (45-60 min)

Phase 1: Scope Definition (5 min)

Interviewer: "Design the document editing service"

Q: Which part — metadata, real-time editing, or versioning?

A: Core editing flow: WebSocket handler + OT engine + persistence

Q: Include OT algorithm internals?

A: Yes — walk through the transform logic

Q: Include versioning reconciliation?

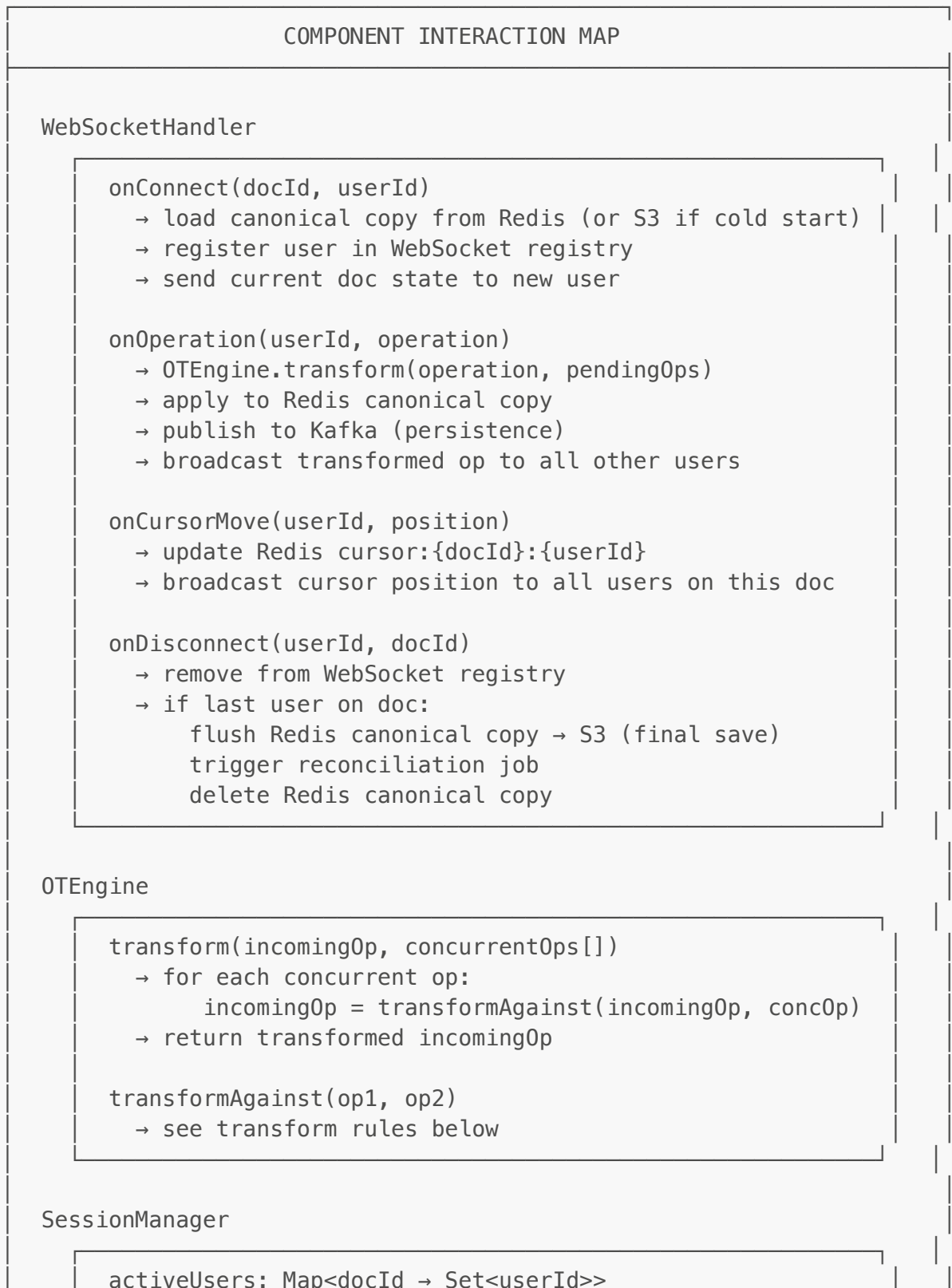
A: Yes — the session lifecycle

Q: Language/framework?

A: Describe in pseudocode/diagrams – implementation–language agnostic

Phase 2: Component Interaction Map (15 min)

Draw on whiteboard:



```
pendingOps: Map<docId → Queue<Operation>>  
isLastUser(docId, userId) → boolean
```

Phase 3: Data Models (10 min)

Operation (the core unit):

Operation Entity	
Field	Description
operationId	Unique ID (Snowflake)
documentId	Which document this op belongs to
userId	Who made this change
type	INSERT / DELETE / RETAIN
position	Where in the document (character index)
character	What character (for INSERT)
length	How many chars (for DELETE)
version	Document version this op was based on
serverVersion	Server-assigned version after transform
timestamp	When the op was created

Why "version" field matters:

Server uses version to know which concurrent ops to transform against.
op.version = 5 means "I made this change based on doc state at v5"

If server is at v7, ops at v6 and v7 happened concurrently → transform against both.

Document Metadata:

Document (Cassandra)	
documentId	UUID (partition key)
title	Document title
ownerId	Who created it
blobUrl	S3 URL of latest version
currentVersion	Latest version number
createdAt	Timestamp
updatedAt	Last modification timestamp
collaborators	Set of user IDs with access

Version Table:

Version (Cassandra)	
documentId	Partition key
versionId	Clustering key (time-ordered)
versionNumber	10, 10.1, 10.2, 11 ...
blobUrl	S3 URL of this version's snapshot
createdBy	Who triggered this version
createdAt	When
isMajor	true for reconciled versions, false for auto

Redis Keys:

REDIS KEY SCHEMA	
canonical:{docId}	Full text of document (string) TTL: refresh while editors active
version:{docId}	Current server version number (int) Incremented on each applied op
cursor:{docId}:{userId}	Cursor position (int) TTL: 30s (refreshed on movement)
activeUsers:{docId}	Set of userIds currently editing Used to detect "last user" condition

Phase 4: OT Engine — Walk Through the Logic (10 min)

"Let me walk through the OT engine logic with all the cases."

Operation Types:

OPERATION TYPES
INSERT: { type: INSERT, pos: 3, char: "X" } → Insert character X at position 3
DELETE: { type: DELETE, pos: 5, length: 2 } → Delete 2 characters starting at position 5

```
RETAIN: { type: RETAIN, length: 10 }
→ Keep next 10 characters unchanged
→ Used in batch operations (OT over whole doc)
```

Transform Rules — Full Decision Tree:

```
transformAgainst(op1, op2):
  (transform op1 to account for op2 having already happened)
```

Case 1: op2 is INSERT at pos P

If op1 is INSERT:

op1.pos > P → op1.pos = op1.pos + 1

op1.pos < P → op1.pos unchanged

op1.pos = P → tie-break by userId (lower ID wins)

If op1 is DELETE:

op1.pos >= P → op1.pos = op1.pos + 1

op1.pos < P → op1.pos unchanged

Case 2: op2 is DELETE at pos P

If op1 is INSERT:

op1.pos > P → op1.pos = op1.pos - 1

op1.pos <= P → op1.pos unchanged

If op1 is DELETE:

op1.pos > P → op1.pos = op1.pos - 1

op1.pos = P → op1 = NOOP (same char deleted by op2)

op1.pos < P → op1.pos unchanged

Server Operation Flow:

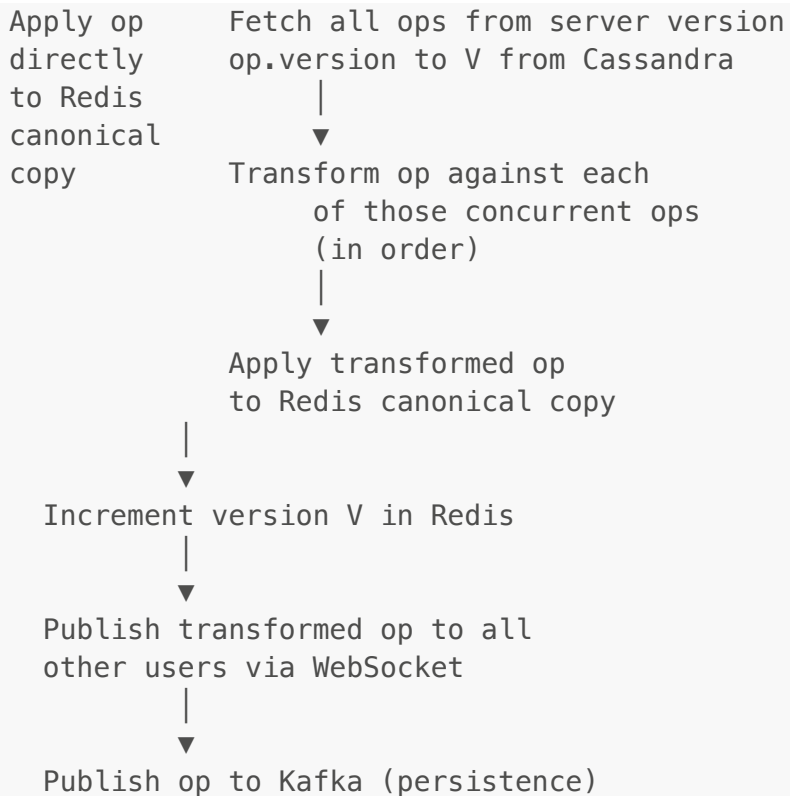
Server receives op from User A:

Read current server version V from Redis

op.version == V?

YES

NO (concurrent ops happened)



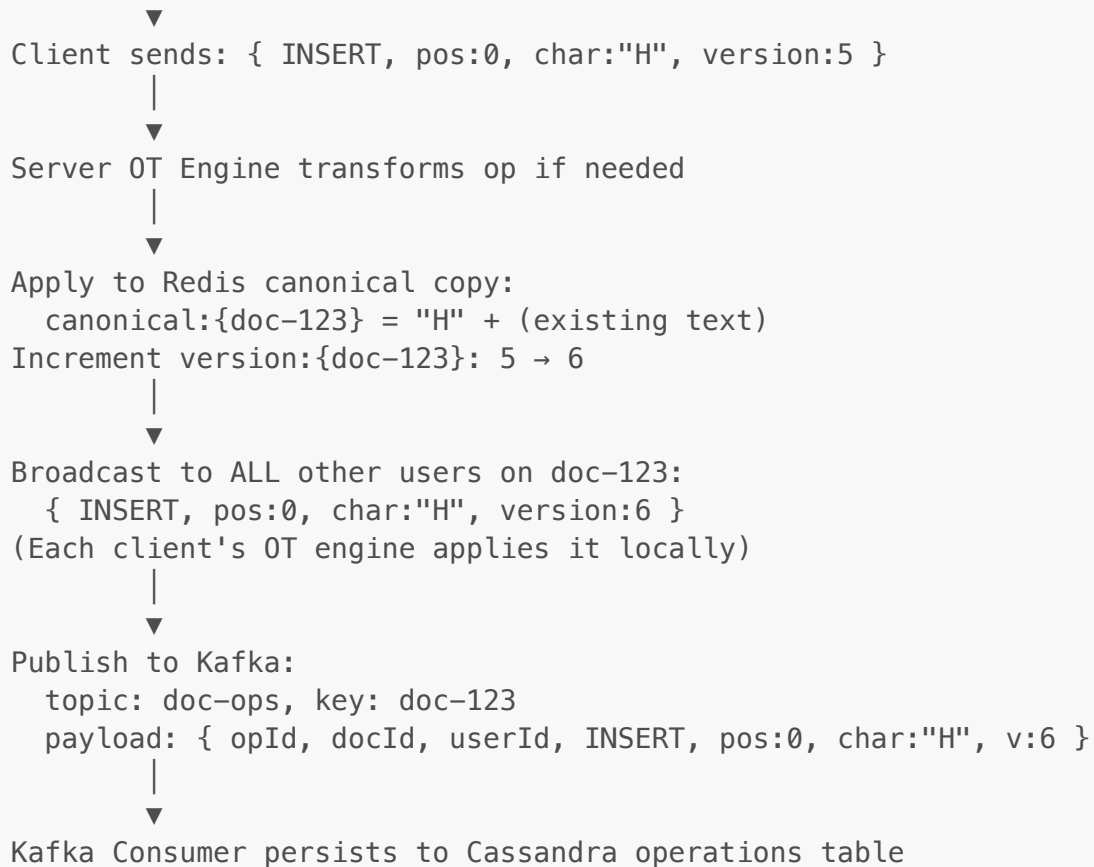
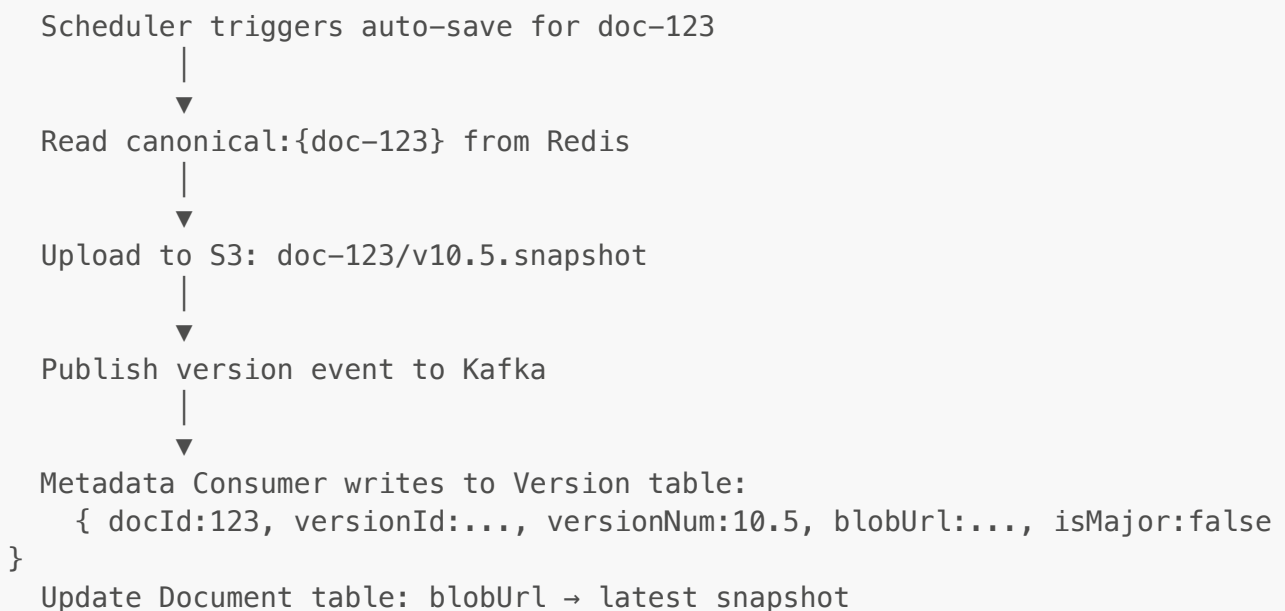
Phase 5: Document Lifecycle — Full Flow (10 min)

User Opens Document:



User Edits Document:

User A types "H" at position 0

**Auto-Save (every 10-20 seconds):****Session End — Reconciliation:**

Last user disconnects from doc-123



RECONCILIATION JOB

```
Step 1: Fetch base version from S3
        (the major version before this session started)

Step 2: Fetch all ops from Cassandra
        WHERE docId = 123 AND timestamp >= session_start

Step 3: Replay all ops on base version
        (produces final document state)

Step 4: Upload final state to S3 as new MAJOR version
        doc-123/v11.snapshot

Step 5: Update Document table: currentVersion = 11

Step 6: DELETE minor versions v10.1 through v10.50 from S3
Step 7: DELETE all session ops from Cassandra
Step 8: DELETE canonical:{doc-123} from Redis
```

Versioning progression:

```
Before session: v10 (major) exists in S3
During session: v10.1, v10.2, ... v10.50 (minor, auto-save)
After reconcile: v11 (major) created, all minor DELETED

Storage saved: 50 minor snapshots deleted, only 2 versions
(v10 and v11) remain
```

Phase 6: Design Patterns (5 min)

1. Command Pattern — Operations as first-class objects

Every edit is an Operation object:

```
Operation {
  apply(document)    → executes the op on a document
  transform(op2)      → returns transformed version
  invert()            → returns the undo operation
  serialize()         → JSON for Kafka/storage
}
```

```
InsertOperation extends Operation { ... }
```

```
DeleteOperation extends Operation { ... }
```

Benefit: undo/redo is just replaying inverted ops

Benefit: persistence is just serializing the queue

2. Observer Pattern — Broadcast to collaborators

```
DocumentSession (Subject):
  activeConnections: Map<userId → WebSocketSession>
  subscribe(userId, session)
  unsubscribe(userId)
  broadcast(op, excludeUserId) ← sends to all EXCEPT sender
```

Flow:

User A sends op → OT Engine transforms → DocumentSession.broadcast()

```
DocumentSession broadcasts to B, C, D (not A)
Each client's OT engine applies transformed op
```

3. Strategy Pattern — OT vs CRDT

```
ConflictResolutionStrategy (interface):
  transform(op, concurrentOps[]) → transformedOp
```

```
OTStrategy implements ConflictResolutionStrategy
```

```
CRDTStrategy implements ConflictResolutionStrategy
```

DocumentEditorService uses strategy at construction time.

Swap algorithm without changing rest of codebase.

Future: switch to CRDT for offline support → zero other changes.

4. Event Sourcing — The Ops Log IS the source of truth

Traditional: Store current state. Done.
 Event Sourcing: Store ALL events (ops). State = replay.

Ops log in Cassandra = event store
 Reconciliation job = state rebuild from events
 Version snapshot = materialized view (optimization)

Benefits:

- Perfect audit trail
- Undo/redo: just replay ops in different order
- Debugging: replay exact sequence to reproduce bug
- Version history: replay up to timestamp T

Phase 7: Cursor Position System (5 min)

"Cursor tracking is often overlooked but asked about. Let me walk through it."

```

User A moves cursor to position 47 in doc-123
    |
    ▼
Client sends cursor event via WebSocket:
{ type: CURSOR_MOVE, userId: "alice", docId: "123", pos: 47, color:
"#FF6B6B" }
    |
    ▼
Server stores in Redis:
SETEX cursor:{doc-123}:{alice} 30 "47" ← TTL 30s (refreshed on
movement)
    |
    ▼
Broadcast to ALL other users on doc-123:
{ type: CURSOR_UPDATE, userId: "alice", pos: 47, color: "#FF6B6B" }
    |
    ▼
Each client renders Alice's cursor at pos 47 with her color
  
```

Cursor position shifts automatically when ops arrive

Alice's cursor is at pos 47
 Bob inserts a char at pos 3 (before Alice's cursor)
 → Alice's cursor must shift to pos 48

This transform is done CLIENT-SIDE using the same OT rules:
 If INSERT at pos $P < \text{cursor.pos}$ → $\text{cursor.pos} += 1$
 If DELETE at pos $P < \text{cursor.pos}$ → $\text{cursor.pos} -= 1$

Why TTL on cursor keys?

- If user's connection dies, cursor disappears after 30s
- No stale "ghost cursors" haunting the document

Common Questions & Answers

Q1: What if two users insert at the exact same position?

Tie-breaking strategy:

Both Alice and Bob insert at position 5 simultaneously.

Without tie-breaking: non-deterministic order → different clients show different results → inconsistency.

Solution: Tie-break by userId (lexicographic / hash order)

If Alice.userId < Bob.userId (alphabetically):
Alice's char goes first (lower position wins)
Bob's char goes one position after

This is deterministic: every client computes the SAME result
because the rule is: lower userId = lower position

Q2: How do you implement Undo/Redo?

Event Sourcing makes this elegant:

Each operation has an invert():

INSERT "H" at pos 0 → invert = DELETE at pos 0, length 1

DELETE at pos 5, len 2 → invert = INSERT deleted chars at pos 5

Undo stack (per user):

Client maintains: undoStack[] and redoStack[]

User presses Ctrl+Z:

op = undoStack.pop()

inverseOp = op.invert()

send inverseOp to server (goes through OT engine)

redoStack.push(op)

Challenge: concurrent edits invalidate undo history

Solution: transform undo ops against concurrent ops

```
(same OT transform rules apply to undo ops)
```

Q3: How do you handle a user going offline mid-edit?

With OT (Google's approach):

```
User goes offline at server version 5
User makes 20 ops locally while offline
User reconnects – server is now at version 47

Client sends all 20 ops with base version = 5
Server transforms each op against versions 6–47
(42 transforms to compute – expensive for long offline)

This is why OT handles offline poorly:
The longer offline, the more transforms needed
```

With CRDT (Notion/Figma approach):

```
User goes offline – makes 20 ops locally
CRDT assigns unique fractional positions to each char
User reconnects – sends all 20 ops
Server merges by fractional position (no transformation!)
O(1) per op regardless of offline duration

CRDT is strictly better for offline scenarios
```

Q4: How does document sharing and access control work?

Access Control Model:

```
Document has an ACL (Access Control List):
  owner:    {userId: alice, role: OWNER}
  editors:  [{userId: bob, role: EDITOR}, ...]
  viewers:  [{userId: carol, role: VIEWER}, ...]
  link:     { type: ANYONE_WITH_LINK, role: COMMENTER }

On WebSocket connect:
  1. Validate JWT token
  2. Check ACL for document access
  3. If VIEWER: send doc but reject any edit ops
  4. If EDITOR: full read/write
```

Share Link generation:

```
POST /documents/{docId}/share
→ Generate short token (UUID truncated)
→ Store { token → docId, role, expiry } in Redis
→ Return share URL: docs.example.com/d/{token}
```

On access via token:

```
→ Lookup token in Redis → get docId and role
→ Create temporary session with that role
```

Q5: How do you scale beyond 100 concurrent editors per document?

100 concurrent editors is already extreme (most docs have 1–5).
But at extreme scale:

Approach 1: Region-based sharding (for OT)

```
Group users by network region (US-East, EU-West)
Each region has an OT server for the document
Regions sync with each other every 100ms
(Jupiter protocol – Google's actual approach)

Trade-off: slight divergence between regions (100ms lag)
```

Approach 2: Switch to CRDT at scale

```
CRDT doesn't need a central server
Peer-to-peer sync: each client syncs with neighbors
No single bottleneck
Trade-off: storage grows (every char has a position ID)
```

Q6: WebSocket server crashes mid-session — what happens to in-flight ops?**Scenario:**

```
5 users editing doc-123 on WS Server 3.
Server 3 crashes.
In-flight ops (sent but not acknowledged) are lost.
Redis canonical copy on Server 3's local memory is gone.
(Redis is a separate cluster – it survives the WS server crash)
```

Recovery flow:

```
Step 1: Client detects disconnect
WebSocket onClose fires → client enters "reconnecting"
Client holds a local queue of unacknowledged ops
```

(every op sent gets an ack; unacked ops stay in queue)

Step 2: Client reconnects to new WS server

Load balancer routes to WS Server 5 (consistent rehash)

Client sends: { reconnect, docId, lastSeenVersion: 47 }

Step 3: New server loads canonical copy from Redis

canonical:{doc-123} still in Redis (separate cluster)

New server fetches version:{doc-123} = 51 (others wrote)

Step 4: Server sends client the ops it missed

Fetch ops v48, v49, v50, v51 from Cassandra ops log

Send to client: "here's what happened while you were gone"

Step 5: Client replays missed ops, then resubmits queue

Client OT engine transforms queued ops against v48-v51

Resubmits transformed ops to new server

What if Redis also lost the canonical copy? (see Q7)

What's the reconnect window? < 1 second in practice.

Clients show "Reconnecting..." UI briefly – acceptable.

Q7: Redis loses the canonical copy mid-session — data loss?

Scenario:

Document actively being edited.

Redis node holding canonical:{doc-123} fails.

Last auto-save to S3 was 15 seconds ago.

15 seconds of unsaved ops exist only in the ops log (Cassandra).

Recovery:

No data loss – because of the ops log in Cassandra.

Every applied op is published to Kafka synchronously before the ack is sent to the client.

Kafka Consumer → Cassandra ops log = durable before ack

Recovery sequence:

1. Detect Redis miss: GET canonical:{doc-123} = nil
2. Fetch last major version from S3 (say v10)
3. Fetch all ops AFTER v10 from Cassandra (last 15 seconds of ops)
4. Replay ops on v10 → reconstruct canonical copy
5. Reload into Redis
6. Resume editing – zero data lost

Why this works:

Ops log in Cassandra is the source of truth (event sourcing).
Redis canonical copy is a materialized view – always rebuildable.
"Redis holds working state, Cassandra holds proof."

Latency of recovery:

Fetch v10 from S3: ~100ms
Fetch 15s of ops (at 5 ops/sec × 100 users = 7500 ops): ~50ms
Replay 7500 ops: ~20ms
Total: ~170ms – users see a brief "reconnecting" flash.

Q8: Reconciliation job fails — what happens?

Scenario:

Session ends. Last user disconnects.
Reconciliation job is triggered.
Job fails halfway (OOM, network partition, etc.)

Problems if not handled:

Minor versions (v10.1 ... v10.50) pile up in S3 forever.
Ops in Cassandra never deleted → storage grows unbounded.
Version table never gets the major version entry.

Defense:

Step 1: Make reconciliation idempotent
Job checks: does major version for this session exist?
If yes: already reconciled – skip
If no: run reconciliation
Safe to retry N times without side effects

Step 2: Retry with exponential backoff
Job fails → retry in 10s, 30s, 2min, 10min
Dead-letter queue after 5 failures → alert on-call

Step 3: Independent cleanup job (safety net)
Runs daily: finds docs with minor versions > 7 days old
Triggers reconciliation for them (belt-and-suspenders)

Step 4: S3 lifecycle policy
Minor versions (isMajor=false) auto-expire after 30 days
Even if reconciliation never runs, storage self-heals

Q9: Client reconnects after brief disconnect — version reconciliation

Scenario:

User loses WiFi for 3 seconds.
They typed 5 characters offline.
Server moved from version 10 to version 14 during the gap.
Client reconnects. What happens?

Client's local state:

Last confirmed server version: 10

Unacknowledged ops: [op_a, op_b, op_c, op_d, op_e]

Each op has version = 10 (based on client's last known)

On reconnect, client sends:

{ reconnect: true, docId, lastVersion: 10 }

Server sends back:

Ops v11, v12, v13, v14 (what happened during disconnect)

Client OT engine:

Transform [op_a...op_e] against [v11, v12, v13, v14]

(same transform rules – each client op adjusted for the server ops that happened concurrently)

Client sends transformed ops to server:

Server applies them, advances to v15, v16, v17, v18, v19

Result: seamless merge – user's typed chars appear correctly interleaved with what others typed

This is exactly why OT version numbers matter:

op.version tells the server EXACTLY which ops to transform against.
Without version numbers: server can't determine concurrency.

Q10: Can you run OT in an active-active multi-region setup?

This is the hardest trap question. The answer is: NOT with standard OT.

Why OT requires a single server:

OT relies on TOTAL ORDERING of operations.

All ops must be processed in the same sequence by all participants. The server enforces this sequence.

With two servers (active-active):

Server A processes: [op1, op2, op3]

Server B processes: [op1, op3, op2] (network reordering)

OT transforms depend on order.
Different order → different transformation results
→ Documents DIVERGE across regions
→ INCONSISTENCY (users in US and EU see different docs)

What Google actually does (Jupiter protocol):

Primary server per document session (single region)
All ops flow to that primary server
Replica servers in other regions are READ-ONLY mirrors
Users near the primary get low latency
Users far away have slightly higher latency (50-150ms)
Acceptable for most collaboration scenarios

If you truly need multi-region active-active:

→ Switch to CRDT.

CRDT uses fractional positions – merge is commutative and associative. Order doesn't matter. Both regions converge.
Trade-off: higher storage, no central server needed.

Interview punchline:

"OT gives you simplicity at the cost of centralization.

CRDT gives you distribution at the cost of storage.

For global Google Docs scale, they use OT with regional primary routing – not true active-active."

Q11: How do you prevent a malicious user from spamming 10,000 ops/sec?

Attack: user writes a script to send 10,000 INSERT ops per second.

Impact:

- Saturates the WS server's OT engine
- Floods Kafka with ops
- Other editors on same doc get their ops delayed
- Cassandra ops log bloated

Defense layers:

Layer 1: API Gateway rate limiting
Max 100 ops/sec per userId per document
Token bucket algorithm: burst allowed, sustained limited
Exceeded → op rejected with 429, client backs off

Layer 2: Client-side throttle (cooperative)
Legitimate clients batch keystrokes:
Don't send op per keystroke – batch every 50ms
"HELLO" typed in 200ms = 1 op "INSERT HELLO at pos X"
Reduces ops from 5 keystrokes to 1 batch op

```

Layer 3: WS Server circuit breaker per connection
  If user sends > 200 ops/sec → close WebSocket connection
  Flag userId for review

Layer 4: Op size limit
  Max op payload: 10KB
  Prevents single-op attacks (huge INSERT of 1MB string)

```

Q12: How do you handle very large documents (100MB+)?

Problem:

```

canonical:{docId} in Redis = 100MB string.
Every op requires: READ 100MB → modify → WRITE 100MB.
Even at Redis speeds, 100MB per op × 25M ops/sec = infeasible.

```

Reality check:

```

Google Docs has a 1.02 million character limit per document.
At ~1 byte/char: 1MB max per doc (not 100MB).
Most design questions assume reasonable document sizes.

```

But if asked about large docs (reports, legal contracts):

Solution: Chunk the document

```

Split document into chunks of 10,000 characters each:
canonical:{docId}:chunk:0   (chars 0-9999)
canonical:{docId}:chunk:1   (chars 10000-19999)
...

```

```

Op arrives: INSERT at position 45,200
→ Maps to chunk 4 (positions 40000-49999)
→ Only read/write chunk 4 (10KB, not 100MB)

```

```

Client loads visible viewport only:
User sees page 5 → load chunks 4 and 5 only
Scroll down → lazy-load chunk 6
Virtual rendering: only DOM for visible text

```

```

Op that crosses chunk boundary:
DELETE from pos 9990 to 10010 (spans chunks 0 and 1)
→ Lock both chunks, apply, update chunk indices

```

Cheat Sheet

Key Numbers to Remember

```
DAU:           100M users
Documents:     10 billion total
Active docs:   5M simultaneously edited
Edit events/sec: 25M/sec (25 million!)
Concurrent editors/doc: up to 100
Auto-save interval: every 10-20 seconds
Latency target: < 100ms edit propagation
Redis canonical TTL: while editors active + 30 min buffer
Cursor TTL:    30 seconds (refreshed on movement)
```

Data Storage Quick Reference

```
Redis:    Canonical copy (live doc in memory) + cursor positions + version counter
S3:       All version snapshots (permanent, CDN-cached for reads)
Cassandra: Document metadata, version table, transient ops log
Kafka:    Operation events stream (durability buffer)
CDN:      Read-only document access (caches S3 snapshots)
```

Algorithm Comparison

```
OT (Operational Transformation):
- Transform concurrent operations before applying
- Requires central server (single point of coordination)
- Used by: Google Docs
- Offline support: poor (many transforms on reconnect)
- Complexity: high

CRDT (Conflict-free Replicated Data Type):
- Fractional position IDs – never conflict
- No central server needed (peer-to-peer capable)
- Used by: Notion, Figma, Atom (Teletype)
- Offline support: excellent (just merge on reconnect)
- Complexity: medium (storage grows over time)
```

Document Lifecycle States

```
COLD:      No active editors → doc lives in S3 only
WARM:      Active editors → canonical copy in Redis, ops in Cassandra
SAVING:    Auto-save in progress → minor snapshot going to S3
CLOSING:   Last editor left → reconciliation job running
CLEAN:     Reconciliation done → back to COLD state
```

OT Transform Rules Summary

Transforming op1 against op2:

op2 = INSERT at P:

op1 = INSERT: pos > P → pos++ | pos < P → unchanged | pos = P → tie-break
 op1 = DELETE: pos >= P → pos++ | pos < P → unchanged

op2 = DELETE at P:

op1 = INSERT: pos > P → pos-- | pos <= P → unchanged
 op1 = DELETE: pos > P → pos-- | pos = P → NOOP | pos < P → unchanged

Design Patterns Used

Command:	Operation as a first-class object (apply, transform, invert)
Observer:	DocumentSession broadcasts ops to all connected editors
Strategy:	OT vs CRDT interchangeable behind interface
Event Sourcing:	Ops log = source of truth, snapshots = materialized views
Factory:	OperationFactory.create(type, pos, char) → Operation
Builder:	DocumentSession.builder().withOTEngine().withRedis()...

Must-Know Concepts

OT:	Transform positions based on concurrent operations
CRDT:	Fractional position IDs that never collide
Canonical copy:	In-memory live doc (Redis) – the ground truth during session
Delta updates:	Send only the change (50 bytes) not the full doc (50 KB)
Reconciliation:	At session end, replay all ops on base → create major version
Sticky routing:	Consistent hash on docId → all editors land on same WS server
Event Sourcing:	Store ops, not state; state = apply(ops) on base version

Final Tips

For HLD Interview

1. **Lead with the unique challenge:** "The design-defining number is 25 million edit events per second — this forces WebSocket over REST and in-memory canonical copies."
2. **Walk through the 3 approaches** (file replacement → locking → delta+OT) — shows structured thinking
3. **Name the algorithm:** "Google uses OT, Figma uses CRDT" — specific facts impress interviewers
4. **Mention the sticky routing constraint** proactively — OT's need for a single server is non-obvious
5. **Explain reconciliation** — most candidates forget the session lifecycle cleanup

For LLD Interview

1. **OT transform rules** — know INSERT vs DELETE cases cold, with tie-breaking
2. **Event Sourcing pattern** — the ops log IS the source of truth
3. **Redis key schema** — canonical copy, version counter, cursor positions
4. **Document lifecycle** — COLD → WARM → CLOSING → CLEAN
5. **Version numbering** — major (reconciled) vs minor (auto-save) distinction

Red Flags to Avoid

- ✗ "We'll use REST for real-time updates" — 25M/sec kills REST
- ✗ "Use database locking for concurrent editing" — terrible UX
- ✗ Jumping to OT without explaining WHY (explain delta + conflict problem first)
- ✗ Forgetting that OT requires all users on same server (sticky routing)
- ✗ No mention of session end cleanup / reconciliation
- ✗ "Store all ops permanently" — they need to be deleted after reconciliation

Green Flags to Aim For

- ✓ "25 million edit events/sec is the design-defining constraint..."
- ✓ "OT requires a central server, which is why we need sticky routing via consistent hashing"
- ✓ "Redis holds the canonical copy — it's like the whiteboard everyone writes on"
- ✓ "At session end, the reconciliation job replays all ops and creates a major version"
- ✓ "CRDT would handle offline editing better — it's a trade-off I'd consider for v2"
- ✓ "Cursor positions transform by the same OT rules as edit ops"

Comparison: Google Docs vs WhatsApp vs YouTube

Dimension	Google Docs	WhatsApp	YouTube
Core challenge	Conflict resolution in shared state	Real-time message delivery	Large-scale read (streaming)
Connection	WebSocket (editing) + REST	WebSocket	HTTP
Algorithm	OT / CRDT	None (append-only)	HLS adaptive bitrate
Primary DB	Cassandra (ops + metadata)	Cassandra (messages)	PostgreSQL (metadata)
In-memory layer	Redis (canonical copy)	Redis (presence + routing)	Redis (hot video cache)

Dimension	Google Docs	WhatsApp	YouTube
Storage	S3 (doc versions)	S3 (media files)	S3 (video files)
Hardest part	OT transform logic	Cross-server WS routing	Video transcoding pipeline
Consistency	Strong within session	At-least-once + dedup	Eventual (views)

Good luck with your interview! The key differentiator at senior level is explaining WHY OT needs sticky routing and HOW the reconciliation job keeps storage clean. 🚀

Return to [System Design Index](#)